Computational Intelligence: Overview

Assoc. Prof. Ayman Ghoneim

Operations Research and Decision Support Department
Faculty of Computers and Artificial Intelligence, Cairo University
Contact Email: a.ghoneim@fci-cu.edu.eg

Computational Intelligence (DS313/DS351) - 2026

Previous Lecture

Previous Lecture !!!

Optimization

Optimization:

Function: noun

First Known Use: 1857

An act, process, or methodology of making something (as a design, system, or decision) as fully perfect, functional, or effective as possible; specifically : the mathematical procedures (as finding the maximum of a function) involved in this.

Source: Merriam-Webster Dictionary (http://www.m-w.com/)

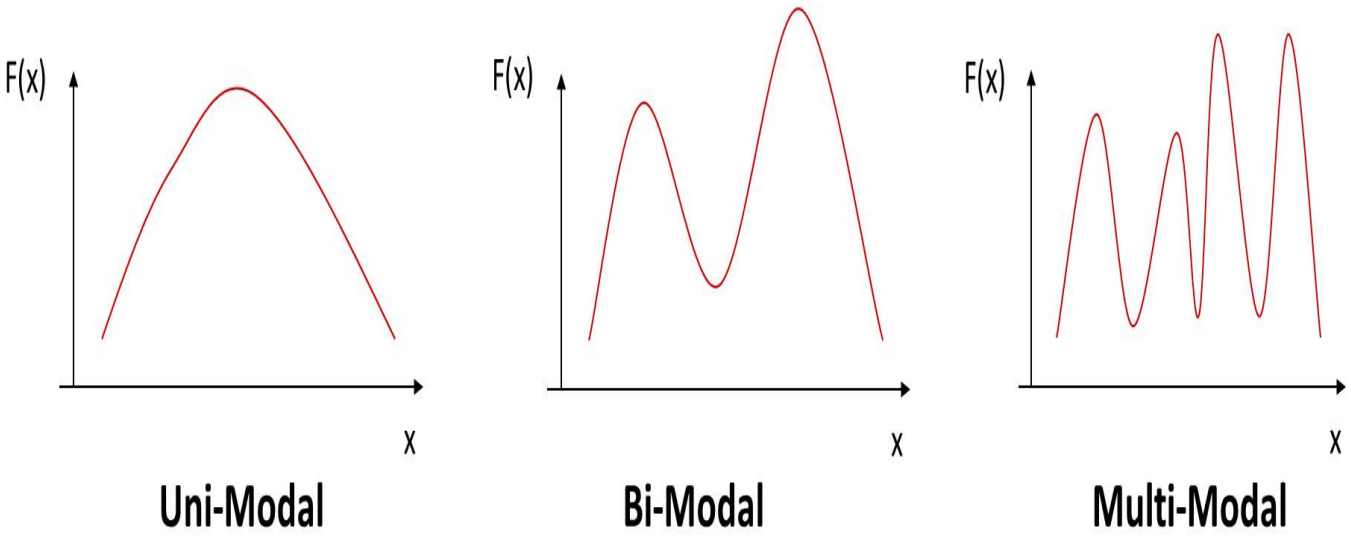
Optimization - Classic Methods

Optimization - Classic Methods

- Generally involves either minimization or maximization.
- Characterized by a well defined objective function.
- Plethora of techniques ranging from calculus based methods to elaborate mathematical optimizers.
- Classical optimization involves the use of gradients to find the next best solution.
- Two main issues that cause problems to classical optimization techniques:
  1. Modality of the search space
  2. Curse of dimensionality

Optimization - Modality

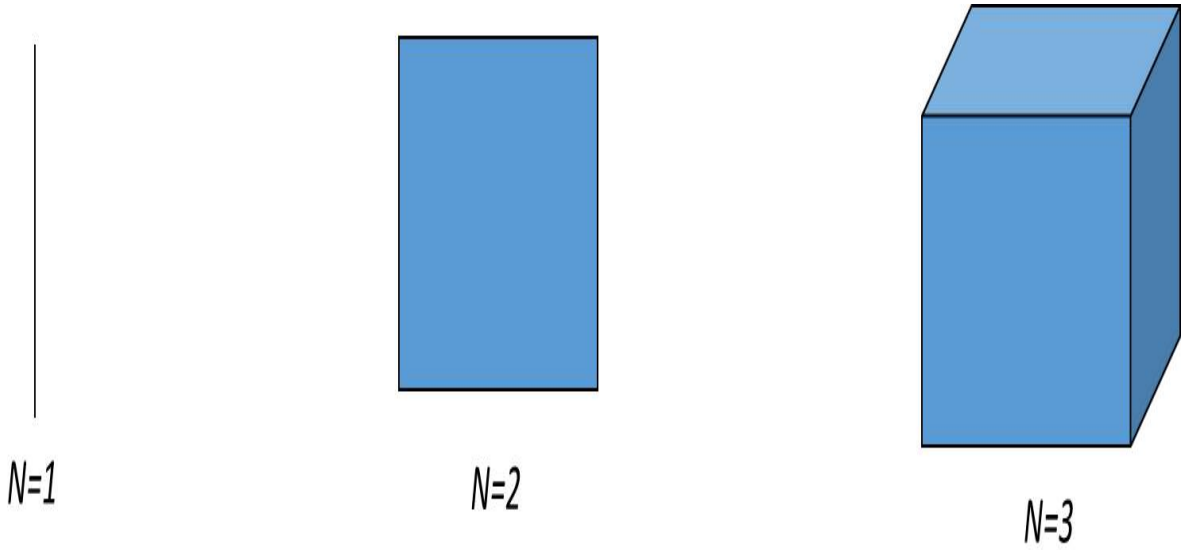
Optimization - Modality of The Search Space



Optimization - Curse of Dimensionality

Optimization - Curse of Dimensionality

- Curse of dimensionality (Bellman 1961) refers to the exponential growth of hypervolume as a function of dimensionality.
- That is, as the number of dimensional increase the volume of space grows exponentially.



Heuristics

Everyday we make tens of decisions, sometimes we think about them carefully, but other times, we make them on the fly using little information or previous experiences (life will be exhausting otherwise !!!).

Heuristics:

A mental shortcut that helps us make judgments and decisions quickly.

They are straight forward rules of thumb which we develop based on previous experiences, e.g.,

- What to wear ? (default outfits)
- What to eat ? (what was enjoyable in the past)

Heuristics are not about making perfect decisions, they are about good decisions quickly.

Introduced by Nobel-prize winner economist Herbert Simon during 1950's.

Heuristics

We need to keep in mind that heuristics may lead to bad decisions.

Types of Heuristics we use:

- Availability heuristic: A mental shortcut which helps us make a decision based on how easy it is to bring something to mind, e.g.,
  - Is the world more violent than before ?
  - Do violent crimes (e.g., murder, robbery, assault) represent the majority of crimes ?
- Representativeness/Base-rate heuristics: A mental shortcut which helps us make a decision by comparing to our mental prototypes / probabilities, e.g.,
  - A young girl how likes reading, will she grow up to be a nurse or a librarian ? (people tend to use Representativeness more than Base-rate heuristics)
  - A patient tests positive for a rare disease (one that affects 1 in every 10000 humans).

Heuristics

We need to keep in mind that heuristics may lead to bad decisions.

Types of Heuristics we use:

- Affect heuristics: A mental shortcut which helps us make a decision based on momentary emotions, e.g.,
  - See decisions with benefits or low risk when in a positive mood, or focus on downsides when in a negative mood.

Optimization - Heuristic Methods

Optimization: The rise of Heuristic Methods

Heuristics 30 or 40 years ago was considered "an admission of defeat", but these days heuristics are considered a respectable area of work.

For "easy" search problems (optimization) we have several well-studied tools:

- calculus,
- linear programming/integer programming,
- calculus of variations,
- dynamic programming.

(Note: Heuristic Search and Heuristic Optimization are often used interchangeably.)

Optimization - Heuristic Methods

Optimization - Heuristic Methods (C. Reeves, 1995):

A heuristic is a technique which seeks "good" (i.e. close to optimal) solutions at a reasonable computational cost without being able to guarantee either feasibility or optimality, indeed in most cases one can not even guarantee how close a heuristic solution is to a global optima.

Optimization - Heuristic Methods

Optimization - Heuristic Methods used for

- Assignment Problems
- Knapsack Problem
- Traveling Salesman Problem
- Hamiltonian Path and Cycle Problem
- Vehicle Routing Problem
- Graph Bisection/Graph Partitioning Problem
- Network Flow Problems
- and many more.

Optimization - Heuristic Methods Examples

Optimization - Heuristic Methods Examples

- Hill Climbing/Adaptive Walk
- Gradient Search and De-Randomized Strategies
- Tabu Search
- Extremal Optimization
- Simulated Annealing/Simulated Tempering
- Simulated Evolution
- Stochastic Evolution (with search cost)
- Genetic Algorithms
- Ant Colony Algorithms
- Particle Swarm Optimization

Hill Climbing

Hill climbing is a mathematical optimization technique that performs local search.

It is an iterative algorithm that starts with an arbitrary initial solution x to a problem, then finds a new solution by searching the neighborhood N and determining the best solution n in the neighborhood.

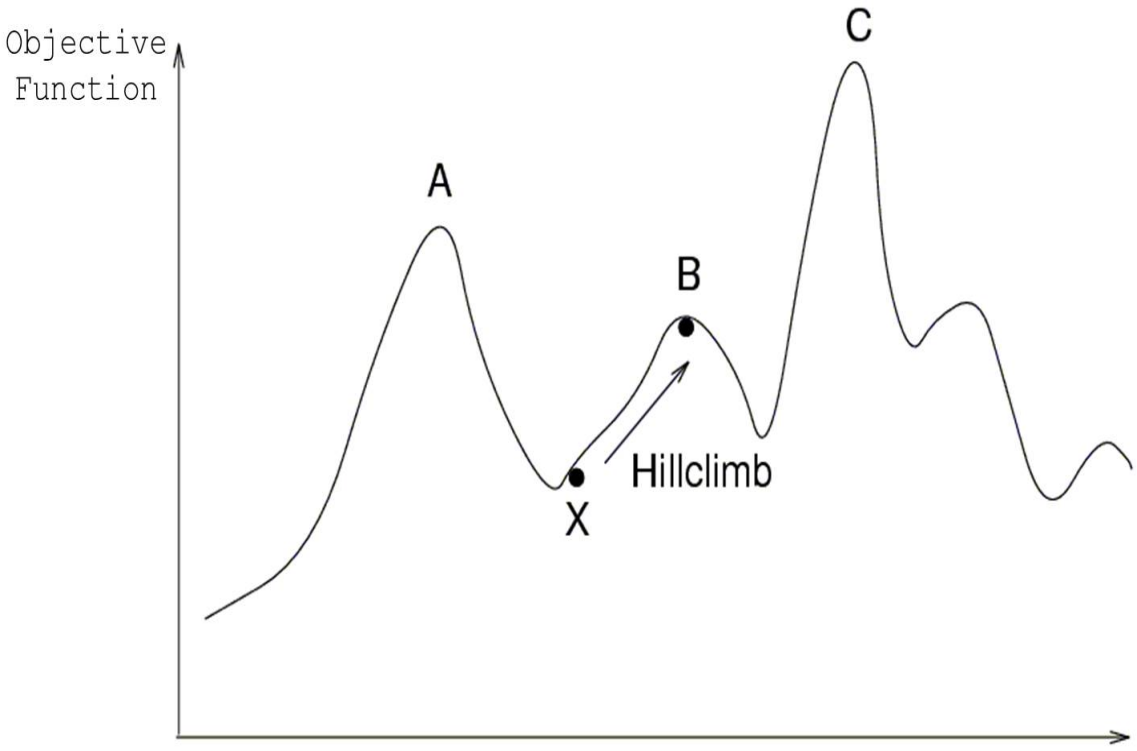
If the new solution is better than the current solution, then the new solution will be considered instead of the current solution and the previous process is repeated again until no further better solution can be found.

Hill Climbing

Algorithm 1 Hill Climbing - Maximization

Input: Initial solution x and a neighborhood function $N(x)$ which produces neighbor points of x .
Repeat:
if $f(n) > f(x)$ **then**
    $x \leftarrow n$
else
    Stop.
end if
Output: x

Hill Climbing



Gradient Descent Search

Gradient descent (also known as steepest descent) is a first-order iterative optimization algorithm for finding local optima of a function.

To find a local minimum of a function using gradient descent, one takes steps proportional to the negative of the gradient of the function at the current point.

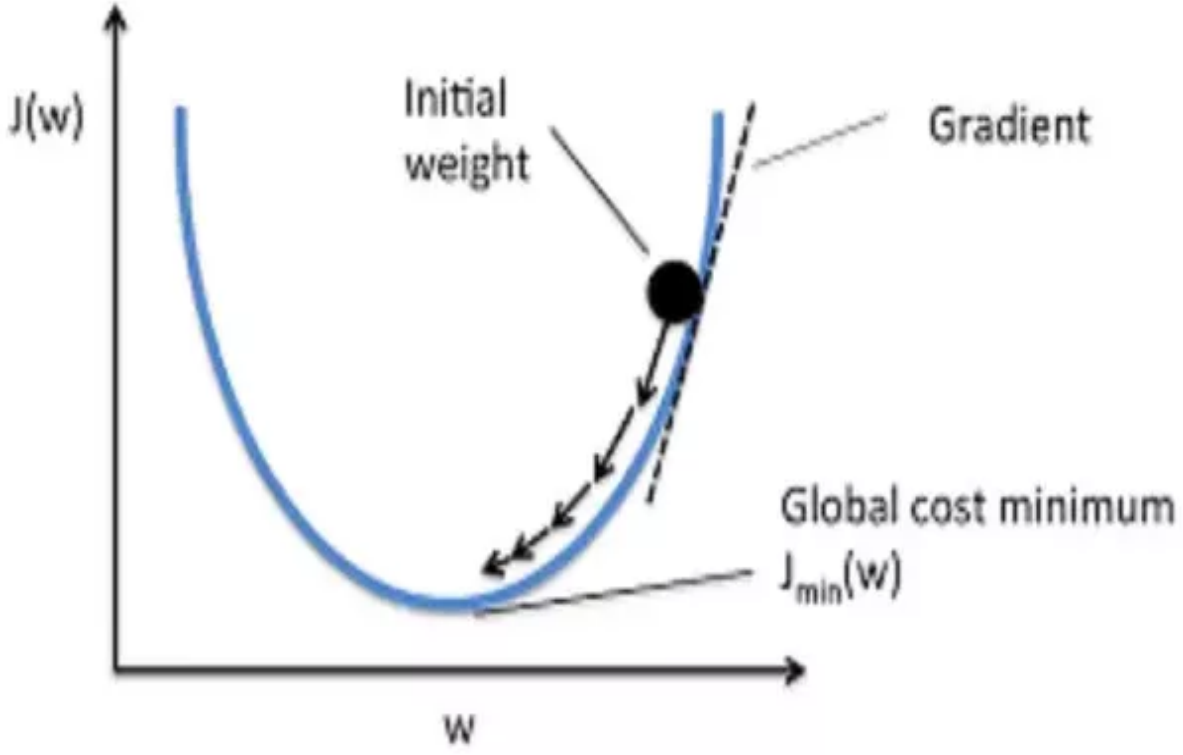
For maximization (finding local maximum), instead, steps are taken proportional to the positive of the gradient. The procedure is then known as gradient ascent.

The step length is controlled by a learning parameter δ.

Algorithm 2 Gradient Descent - Minimization

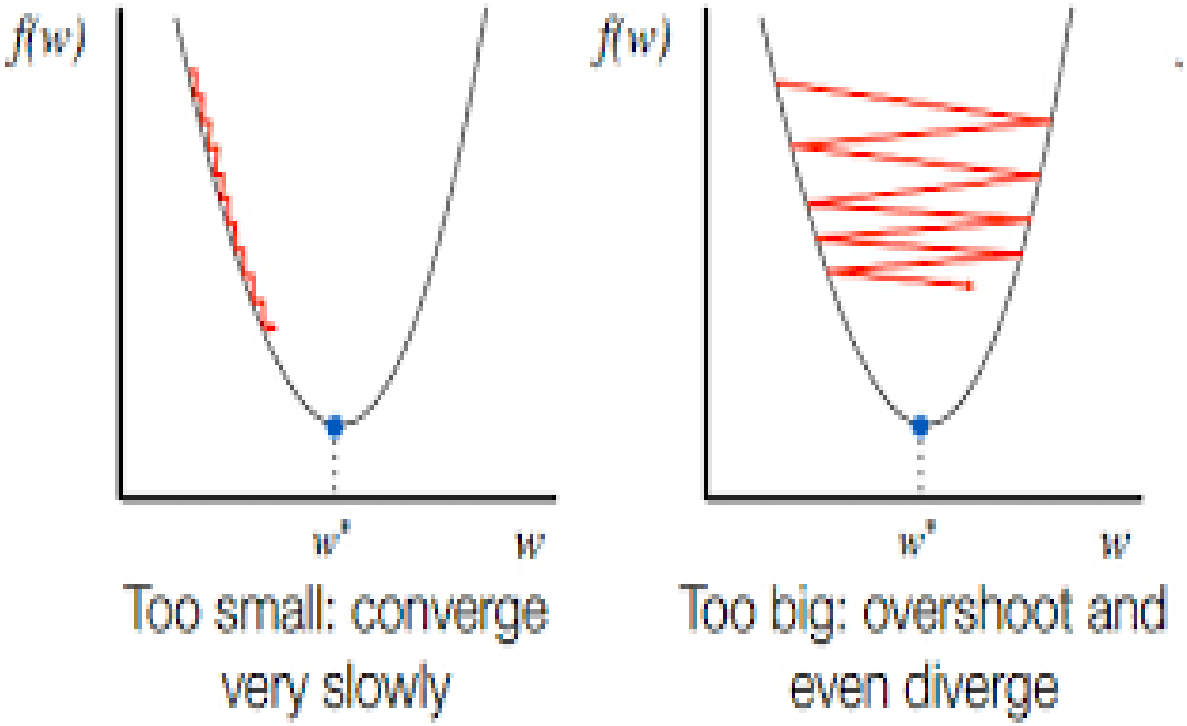
Input: Initial solution x and a parameter δ.
Repeat until no change in x value.
$x \leftarrow x - \delta f'(x)$
Output: x

Gradient Descent Search

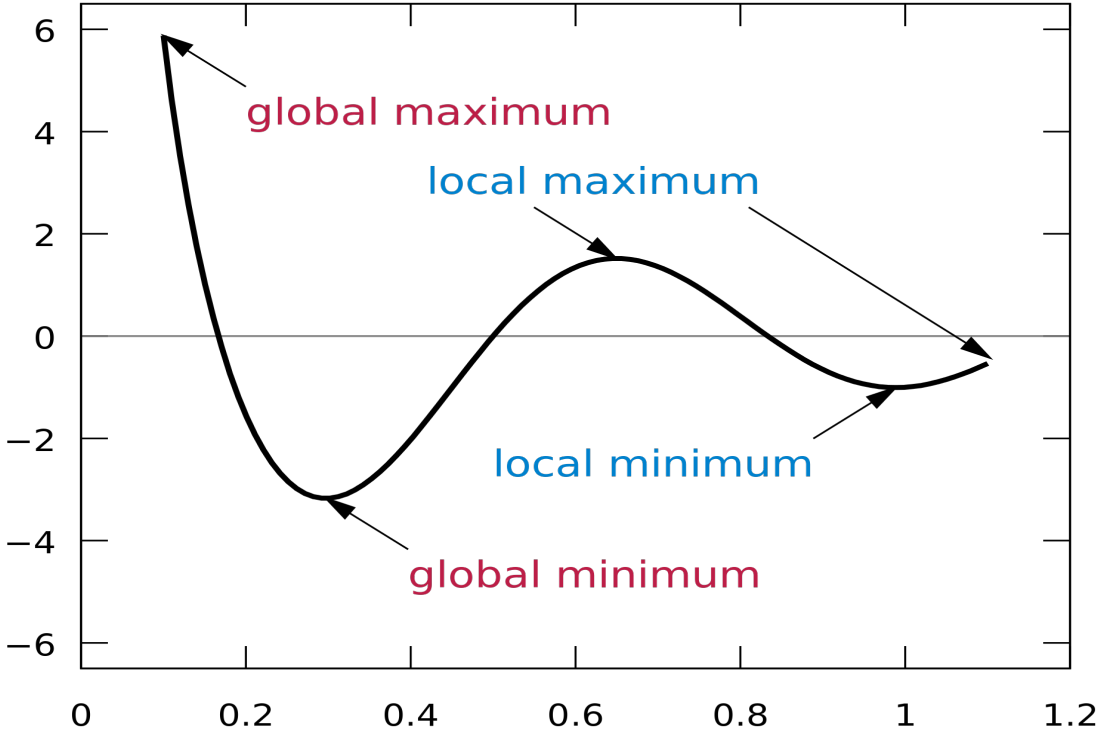


Tunning

The δ parameter needs tunning.

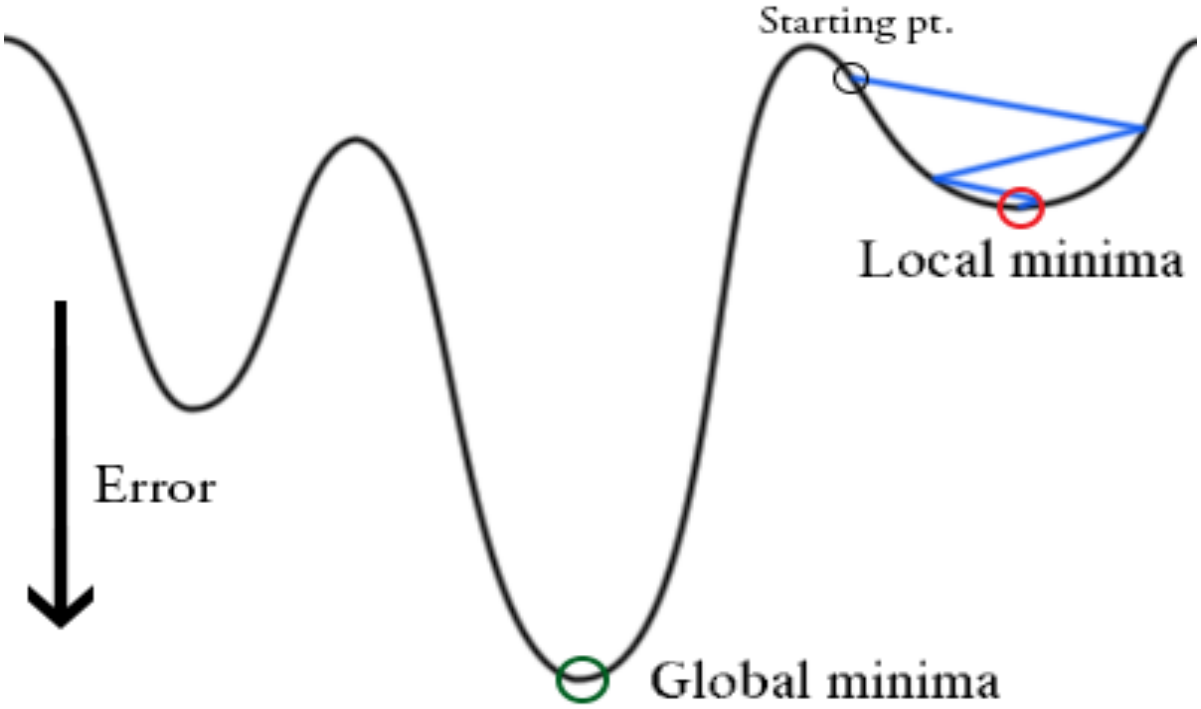


Local Optima



Local Optima

Both Hill climbing and Gradient descent can be trapped in local minimum.

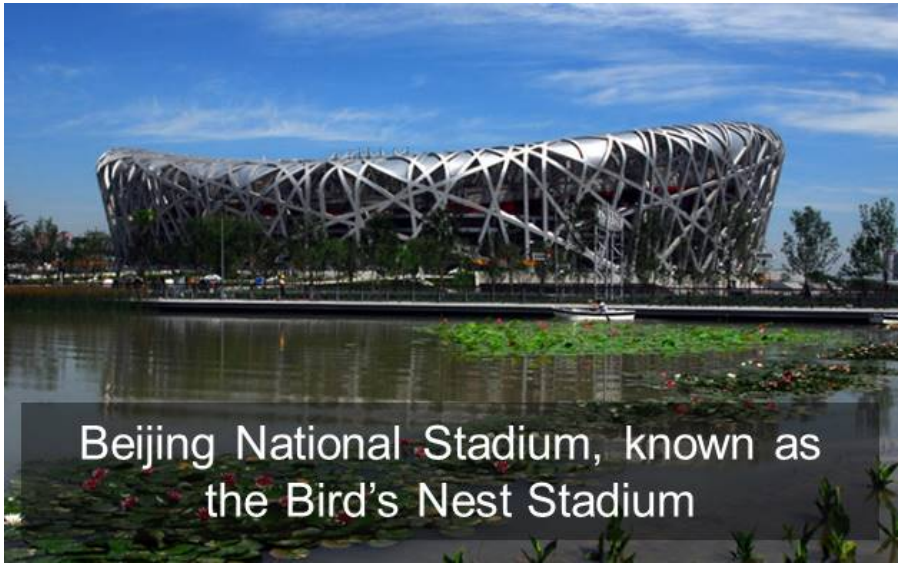


Humans and Mimicking

If you want to do something new, what is the first thing comes to your mind ?

Answer: How others are doing it !!!

Humans are great in mimicking - for elegance.



Humans and Mimicking

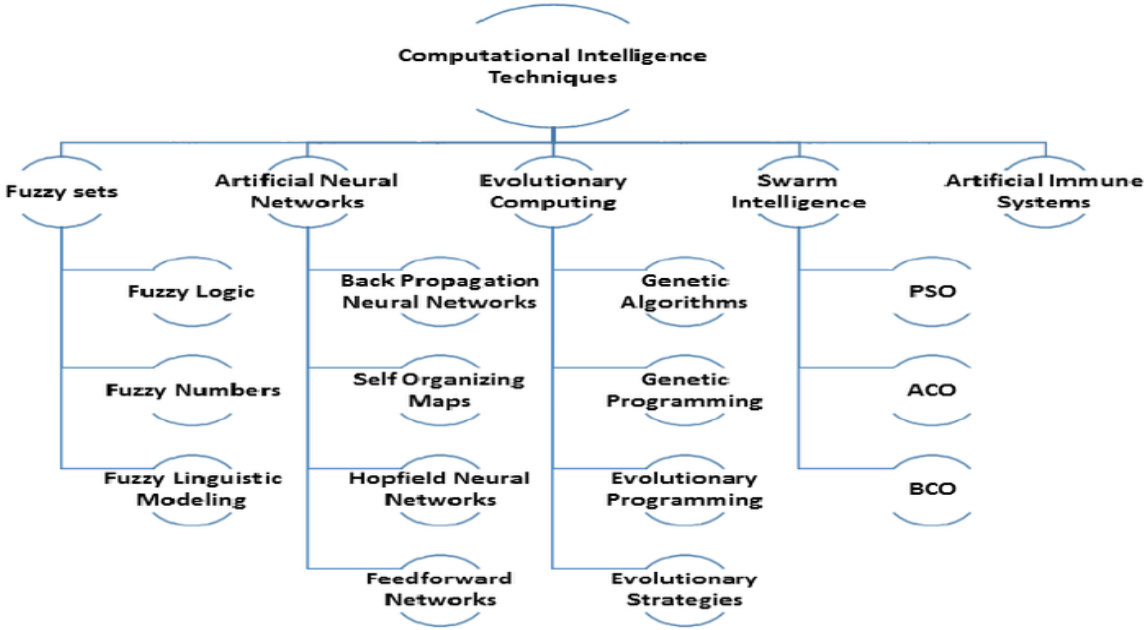
Humans are great in mimicking - for function.

The kingfisher's beak provided an almost ideal shape for a more streamlined Shinkansen - bullet train that travels more quietly (reduced noise resulting from aerodynamics), travels 10% faster, and uses 15% less electricity.

Computation Intelligence - Taxonomy

Computation Intelligence (CI): Building problem solvers by looking at and mimicking nature and humans.

- Metallurgy ⇒ Simulated annealing
- Human Brain ⇒ Neurocomputing
- Biological Evolution ⇒ Evolutionary computing
- Human Fuzzy Thinking ⇒ Fuzzy logic



Simulated Annealing

Simulated Annealing (SA) is an effective and general form of optimization.

"Annealing" refers to an analogy with thermodynamics, specifically with the way that metals cool and anneal.

"Simulated Annealing" name and inspiration come from annealing in metallurgy, a technique involving heating and controlled cooling of a material to increase the size of its crystals and reduce their defects. Both are attributes of the material that depend on its thermodynamic free energy. Heating and cooling the material affects both the temperature and the thermodynamic free energy.

Simulated annealing uses the objective function of an optimization problem instead of the energy of a material.

Analogous:

- The metal configuration C is the solution.
- The energy E of C is the objective function value of a solution.
- The parameter T is the temperature in an annealing system.

Simulated Annealing

SA algorithm is basically hill-climbing except instead of always picking the best move within a neighborhood and going to it (i.e., if the selected move improves the solution, then it is always accepted), SA may accept a worse move with some probability.

Accepting worse solutions is a fundamental property of metaheuristics because it allows for a more extensive search for the global optimal solution.

The difference ΔE in the energy (i.e., quality) between the current point (E_C) and the next point (E_N), in addition to the parameter T is used to determine this probability.

This notion of slow cooling implemented in the simulated annealing algorithm is interpreted as a slow decrease in the probability of accepting worse solutions as the solution space is explored.

At higher values of T, moving to bad points is more likely to occur. As T tends to zero, they become more and more unlikely to happen, until the algorithm behaves like hill-climbing.

Simulated Annealing

In general, the simulated annealing algorithms work as follows:

At each time step, the algorithm randomly selects a solution close to the current one, measures its quality, and then decides to move to it or to stay with the current solution based on either the new solution is better or worse than the current one. A probability is used for the second case.

During the search, the temperature T is progressively decreased from an initial positive value to zero and affects the probability (aslo progressively change towards zero) of accepting a solution with less quality than the current solution.

Simulated Annealing

Algorithm 3 Simulated Annealing Algorithm

Input: T_{max}, T_{min}, and C_{init}.
$C \leftarrow C_{init}$.
for $T = T_{max} to T_{min}$ **do**
    $E_C \leftarrow E(C), N \leftarrow Next(C), E_N \leftarrow E(N)$.
    $\Delta E \leftarrow E_N - E_C$.
    if $\Delta E > 0$ **then**
        $C \leftarrow N$.
    else
        if $\exp^{\frac{\Delta E}{T}} > rand(0,1)$ **then**
            $C \leftarrow N$.
        end if
    end if
end for
Output: C.